
Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Computer Vision and Image Processing (CSCS111/CSDS119)
Lab Assignment 6
Image Segmentation

1 Introduction and Problem Statement

Image segmentation is a foundational task in computer vision aimed at partitioning a digital image into multiple meaningful, visually distinct regions corresponding to objects or structural boundaries. This laboratory assignment explores both classical spatial segmentation approaches and advanced composite pipelines using a real-world photograph (`cup_holding.jpg`) of a person holding an object.

The objectives are structured into two distinct parts:

1. PART A:

- Detect prominent isolated points and directional lines ($0^\circ, 90^\circ, +45^\circ, -45^\circ$) using spatial convolution masks.
- Detect object and human boundaries using spatial edge detection operators: Roberts Cross, Prewitt, Sobel, and Laplacian.
- Segment the image into foreground and background using Iterative Global Thresholding and Otsu's optimal thresholding.
- Implement Region Growing segmentation and systematically analyze the impact of seed point placement and similarity criteria (ΔT).
- Conduct a rigorous comparative evaluation on the relative efficacy of each technique.

2. PART B:

- Design and implement a composite segmentation pipeline to isolate and retain the held object—specifically a **black cup** (intensity values $\approx 30\text{--}70$) held against a **gray background** ($\approx 200\text{--}245$) and hand skin tones ($\approx 150\text{--}210$).
- Remove the surrounding environment (setting non-cup pixels to clean white), or alternately make the object disappear through inpainting while preserving background and hand posture.
- Contrast the pipeline's performance against single-technique thresholding failures (Global Otsu and Naive dark thresholding).

1.1 Input Image Setup and Preprocessing

The test image `cup_holding.jpg` has dimensions 525×350 pixels with 3 RGB color channels. The preprocessing block loads the image using OpenCV, converts color representations, and displays the original input.

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from collections import deque
5
6 # Load image
7 img_bgr = cv2.imread('cup_holding.jpg')
8 img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
9 gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
10
11 print(f"Loaded cup_holding.jpg with dimensions: {img_bgr.shape[1]}x{img_bgr.shape[0]}
12       (Channels: {img_bgr.shape[2]})")
13
14 fig, ax = plt.subplots(figsize=(6, 4))
15 ax.imshow(img_rgb)
16 ax.set_title("Original Image (cup_holding.jpg)")
17 ax.axis("off")
18 plt.tight_layout()
19 plt.show()
```

```
Loaded cup_holding.jpg with dimensions: 525x350 (Channels: 3)
```

Original Image (cup_holding.jpg)



Figure 1: Original input photograph (cup_holding.jpg) showing a person holding a black cup against a neutral gray background.

2 Part A.1: Point and Directional Line Detection

Spatial feature detection can be formulated using localized linear convolution filters. Discontinuities in an image are characterized by abrupt intensity changes across spatial neighborhoods.

2.1 Mathematical Formulation

1. **Point Detection Mask:** A point discontinuity corresponds to an isolated pixel whose intensity differs substantially from all surrounding neighbors. The 3×3 isotropic Laplacian mask is defined as:

$$K_{\text{point}} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} \quad (1)$$

A point is classified as prominent if the absolute filtered response exceeds a predetermined threshold: $|R(x, y)| \geq T_{\text{point}}$.

2. **Directional Line Masks:** Lines of one-pixel thickness oriented along specific angles are detected using masks where coefficients along the preferred direction are positive and orthogonal coefficients are negative:

$$K_H = \begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}, \quad K_V = \begin{bmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{bmatrix}, \quad (2)$$

$$K_{+45^\circ} = \begin{bmatrix} -1 & -1 & 2 \\ -1 & 2 & -1 \\ 2 & -1 & -1 \end{bmatrix}, \quad K_{-45^\circ} = \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix} \quad (3)$$

2.2 Python Implementation

```

1 # Point detection kernel
2 point_kernel = np.array([[ -1, -1, -1],
3                           [-1,  8, -1],
4                           [-1, -1, -1]], dtype=np.float32)
5 point_resp = np.abs(cv2.filter2D(gray, cv2.CV_32F, point_kernel))
6 # Prominent point detection threshold (top 0.5% responses)
7 point_thresh = np.percentile(point_resp, 99.5)
8 point_detected = (point_resp > point_thresh).astype(np.uint8) * 255
9
10 # Directional line detection kernels
11 line_h = np.array([[ -1, -1, -1],
12                   [ 2,  2,  2],
13                   [-1, -1, -1]], dtype=np.float32)
14 line_v = np.array([[ -1,  2, -1],
15                   [-1,  2, -1],
16                   [-1,  2, -1]], dtype=np.float32)
17 line_p45 = np.array([[ -1, -1,  2],
18                     [-1,  2, -1],
19                     [ 2, -1, -1]], dtype=np.float32)
20 line_m45 = np.array([[ 2, -1, -1],
21                     [-1,  2, -1],
22                     [-1, -1,  2]], dtype=np.float32)
23
24 resp_h = np.abs(cv2.filter2D(gray, cv2.CV_32F, line_h))
25 resp_v = np.abs(cv2.filter2D(gray, cv2.CV_32F, line_v))
26 resp_p45 = np.abs(cv2.filter2D(gray, cv2.CV_32F, line_p45))
27 resp_m45 = np.abs(cv2.filter2D(gray, cv2.CV_32F, line_m45))

```

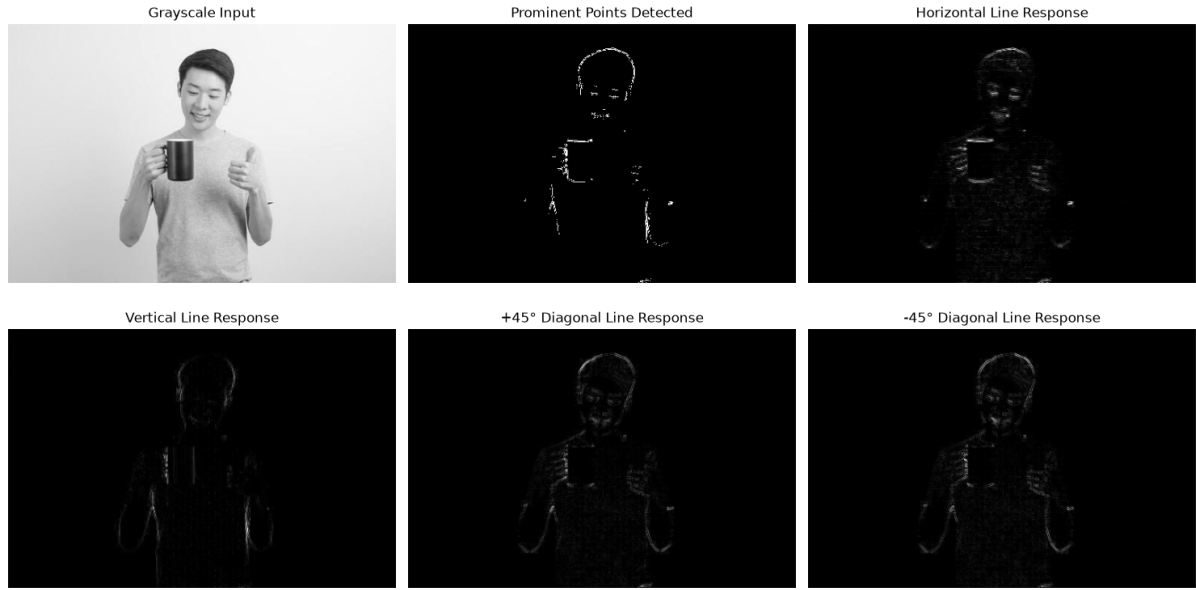


Figure 2: Visual responses for prominent point detection and directional line detection (0° , 90° , $+45^\circ$, and -45°).

Analysis: The point detection mask isolates high-frequency texture anomalies, highlights on the cup rim, and specular reflection spots. The directional line masks selectively amplify linear structures along the specified angle; for instance, the horizontal mask accentuates the shoulders and top of the cup rim, whereas the vertical mask accentuates the sides of the cup and the human torso silhouette. However, line masks produce discontinuous segments and cannot form closed object boundaries.

3 Part A.2: Edge Detection Operators

Edge detection computes spatial gradient approximations to locate boundaries between regions of contrasting intensity.

3.1 Mathematical Formulation

1. **Roberts Cross** (2×2): Computes diagonal finite differences:

$$R_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad R_y = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}, \quad |\nabla f| \approx \sqrt{G_x^2 + G_y^2} \quad (4)$$

2. **Prewitt Operator** (3×3): Uses orthogonal finite differences with equal neighbor weighting:

$$P_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \quad P_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \quad (5)$$

3. **Sobel Operator** (3×3): Incorporates Gaussian smoothing across the center row/column:

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (6)$$

4. **Laplacian Operator:** Isotropic second-derivative operator capturing zero-crossings:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}, \quad L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad (7)$$

3.2 Python Implementation

```
1 # Roberts Cross
2 rob_x = np.array([[1, 0], [0, -1]], dtype=np.float32)
3 rob_y = np.array([[0, 1], [-1, 0]], dtype=np.float32)
4 gx_rob = cv2.filter2D(gray, cv2.CV_32F, rob_x)
5 gy_rob = cv2.filter2D(gray, cv2.CV_32F, rob_y)
6 roberts = np.sqrt(gx_rob**2 + gy_rob**2)
7
8 # Prewitt
9 prew_x = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]], dtype=np.float32)
10 prew_y = np.array([[-1, -1, -1], [0, 0, 0], [1, 1, 1]], dtype=np.float32)
11 gx_prew = cv2.filter2D(gray, cv2.CV_32F, prew_x)
12 gy_prew = cv2.filter2D(gray, cv2.CV_32F, prew_y)
13 prewitt = np.sqrt(gx_prew**2 + gy_prew**2)
14
15 # Sobel
16 sobel_x = cv2.Sobel(gray, cv2.CV_32F, 1, 0, ksize=3)
17 sobel_y = cv2.Sobel(gray, cv2.CV_32F, 0, 1, ksize=3)
18 sobel = np.sqrt(sobel_x**2 + sobel_y**2)
19
20 # Laplacian (2nd Derivative)
21 laplacian = np.abs(cv2.Laplacian(gray, cv2.CV_32F, ksize=3))
```

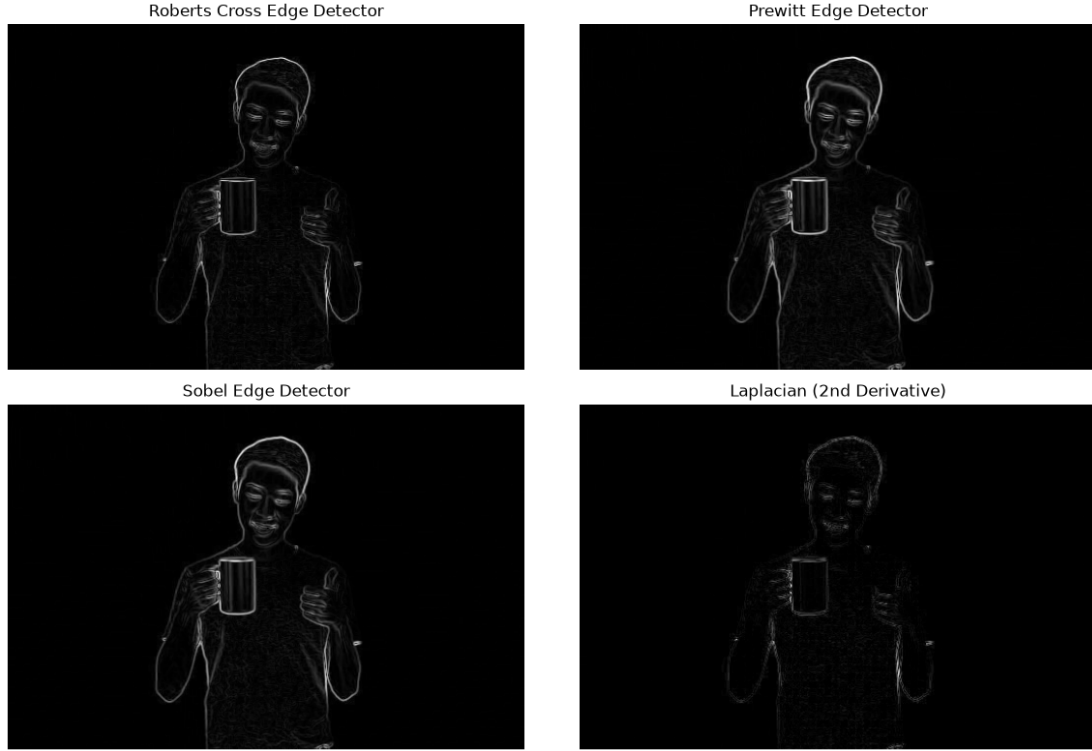


Figure 3: Comparison of spatial edge detection operators: Roberts Cross, Prewitt, Sobel, and Laplacian.

Analysis:

- **Roberts Cross:** Employs a tiny 2×2 stencil. It exhibits the thinnest edge responses but is severely degraded by local pixel noise.
- **Prewitt vs. Sobel:** Both use 3×3 neighborhoods, yet Sobel provides significantly smoother, more continuous edges because the central coefficient (2) incorporates localized triangular/Gaussian smoothing.
- **Laplacian:** As a second-derivative operator, it generates distinct double-edge artifacts and amplifies high-frequency noise across the background.

4 Part A.3: Thresholding-Based Segmentation

Thresholding segments an image by partitioning its intensity histogram into binary classes: foreground (G_1) and background (G_2).

4.1 Mathematical Formulation

1. **Iterative Global Thresholding:** An initial estimate $T_0 = \frac{1}{MN} \sum_{x,y} f(x, y)$ is iteratively refined:

$$m_1 = \frac{1}{|G_1|} \sum_{(x,y) \in G_1} f(x, y), \quad m_2 = \frac{1}{|G_2|} \sum_{(x,y) \in G_2} f(x, y), \quad T_{k+1} = \frac{m_1 + m_2}{2} \quad (8)$$

The loop terminates when $|T_{k+1} - T_k| < \epsilon$ (with $\epsilon = 0.5$).

2. **Otsu's Thresholding:** Selects T^* that maximizes the between-class variance $\sigma_B^2(T)$:

$$\sigma_B^2(T) = \omega_0(T)\omega_1(T)[\mu_0(T) - \mu_1(T)]^2 \quad (9)$$

where ω_0, ω_1 denote cumulative class probabilities and μ_0, μ_1 denote class means up to threshold T .

4.2 Python Implementation and Results

```

1 # Iterative Global Thresholding
2 T = float(gray.mean())
3 while True:
4     g1 = gray[gray > T]
5     g2 = gray[gray <= T]
6     m1 = g1.mean() if len(g1) > 0 else 0
7     m2 = g2.mean() if len(g2) > 0 else 0
8     new_T = (m1 + m2) / 2
9     if abs(T - new_T) < 0.5:
10         break
11     T = new_T
12
13 global_thresh_val = T
14 global_seg = (gray > global_thresh_val).astype(np.uint8) * 255
15
16 # Otsu's Thresholding
17 otsu_thresh_val, otsu_seg = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.
18     THRESH_OTSU)
19
20 print(f"Iterative Global Threshold: {global_thresh_val:.2f}")
21 print(f"Otsu Optimal Threshold:      {otsu_thresh_val:.2f}")

```

```

Iterative Global Threshold: 187.13
Otsu Optimal Threshold:    186.00

```

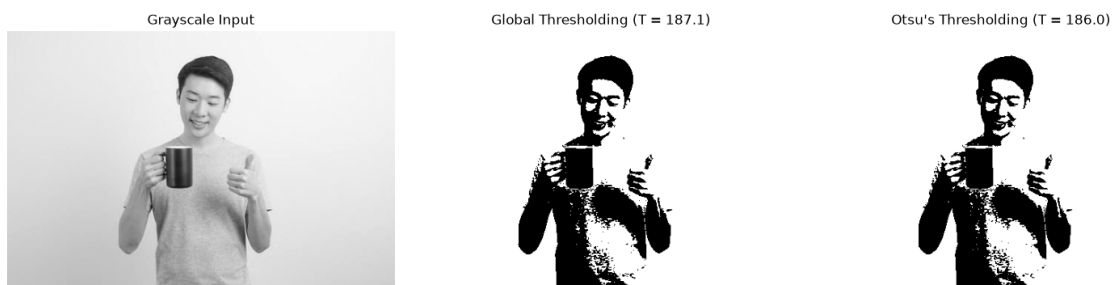


Figure 4: Thresholding comparison: Grayscale input, Iterative Global Thresholding ($T = 187.1$), and Otsu's optimal thresholding ($T = 186.0$).

Analysis: Both algorithms converge to nearly identical partition thresholds (187.13 vs. 186.00). While both succeed in separating the bright gray background from the darker person, **neither method can isolate the black cup alone** because global thresholding bisects the histogram into only two broad classes, grouping the cup, hand, and torso into a single foreground mass.

5 Part A.4: Region-Based Segmentation (Region Growing)

Region growing is a spatial segmentation technique that groups neighboring pixels into larger regions based on predefined similarity criteria.

5.1 Algorithm and Similarity Criterion

Given a seed coordinate $(r_{\text{seed}}, c_{\text{seed}})$, adjacent 4-connected neighbor pixels (r, c) are appended to the region $\mathcal{R}$ if the intensity deviation satisfies:

$$|f(r, c) - f(r_{\text{seed}}, c_{\text{seed}})| \leq \Delta T \quad (10)$$

5.2 Python Implementation and Sensitivity Analysis

```

1 def region_growing(img_gray, seed, threshold=30):
2     h, w = img_gray.shape
3     segmented = np.zeros((h, w), dtype=bool)
4     visited = np.zeros((h, w), dtype=bool)
5
6     sr, sc = seed
7     seed_val = float(img_gray[sr, sc])
8     queue = deque([(sr, sc)])
9     visited[sr, sc] = True
10    segmented[sr, sc] = True
11
12    while queue:
13        r, c = queue.popleft()
14        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
15            nr, nc = r + dr, c + dc
16            if 0 <= nr < h and 0 <= nc < w and not visited[nr, nc]:
17                visited[nr, nc] = True
18                if abs(float(img_gray[nr, nc]) - seed_val) <= threshold:
19                    segmented[nr, nc] = True
20                    queue.append((nr, nc))
21    return (segmented.astype(np.uint8) * 255)
22
23 # Seed 1: Inside Black Cup (row 187, col 230, intensity ~48)
24 seed_cup = (187, 230)
25 cup_t15 = region_growing(gray, seed_cup, threshold=15)
26 cup_t30 = region_growing(gray, seed_cup, threshold=30)
27 cup_t60 = region_growing(gray, seed_cup, threshold=60)
28
29 # Seed 2: On Gray Background (row 50, col 50, intensity ~225)
30 seed_bg = (50, 50)
31 bg_t30 = region_growing(gray, seed_bg, threshold=30)

```

```

Seed at Cup (187, 230) with T=15 -> Segmented Pixels: 442
Seed at Cup (187, 230) with T=30 -> Segmented Pixels: 958
Seed at Cup (187, 230) with T=60 -> Segmented Pixels: 1724
Seed at Background (50, 50) with T=30 -> Segmented Pixels: 139457

```

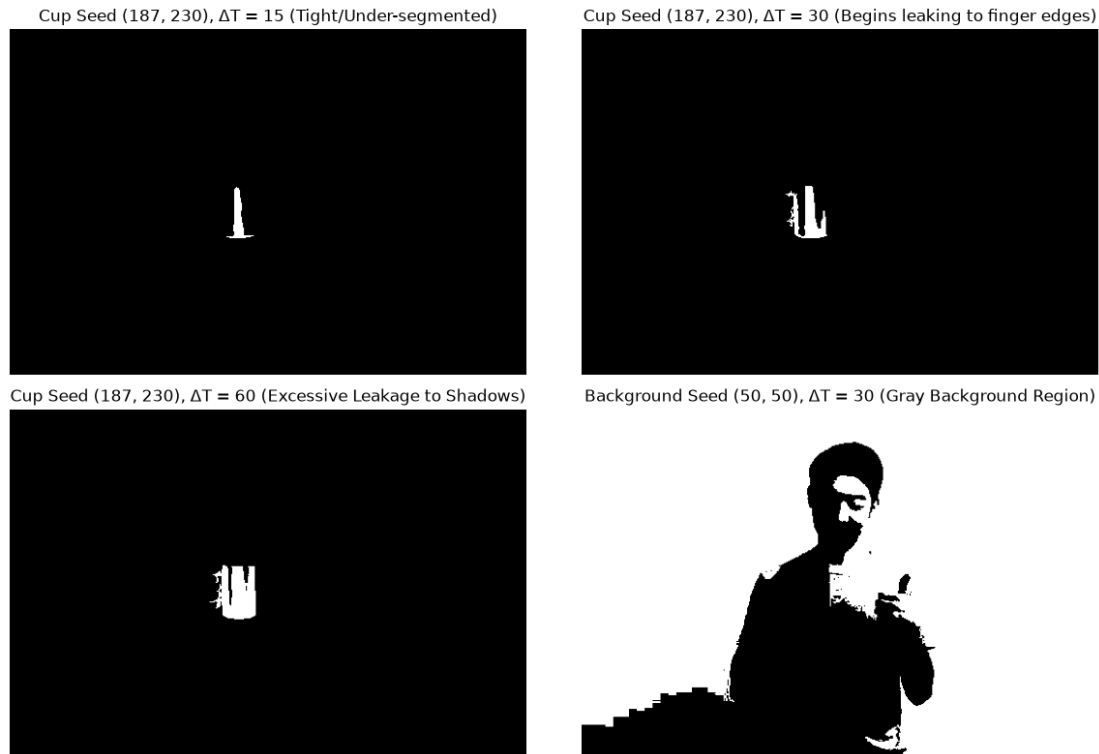


Figure 5: Investigation of Region Growing sensitivity across seed coordinates and similarity tolerances $\Delta T \in \{15, 30, 60\}$.

6 Part A.5: Comparative Observations on Part A Techniques

Table 1 synthesizes the relative operational characteristics of the techniques evaluated in Part A.

Technique	Primary Strength	Limitation in Object Separation
Point Detection	Pinpoints high-frequency impulse noise and highlights.	Cannot segment regions or boundaries.
Line Detection	Selectively highlights linear features at $0^\circ, 90^\circ, \pm 45^\circ$.	Produces broken lines without closed contours.
Roberts Cross	Compact 2×2 stencil, computationally cheap.	Very sensitive to pixel noise; weak boundary continuity.
Prewitt / Sobel	Robust boundary detection; Sobel reduces noise via Gaussian weighting.	Leaves unclosed contours; cannot provide binary region masks.
Global / Otsu	Automates global bimodal partitioning ($T \approx 186$).	Fails when object shares intensity with background or hand.
Region Growing	Most effective local method: spatially constrained to seed.	Highly sensitive to ΔT ; vulnerable to boundary leakage.

Table 1: Comparative evaluation of classical spatial segmentation techniques.

7 Part B: Object Retention and Disappearance Pipeline

7.1 Problem Context: Black Cup vs. Gray Background

In `cup_holding.jpg`, the object held in the hand is a **black cup** (intensity values $\approx 30\text{--}70$), held against a **gray background** ($\approx 200\text{--}245$) and hand skin tones ($\approx 150\text{--}210$). The objective is twofold:

1. **Retain only the black cup** in hand, suppressing the background, person, and hand to pure white.
2. **Make the black cup disappear** from the scene through seamless inpainting while preserving the holding hand and background context.

7.2 Failure Modes of Single Techniques

1. **Failure Mode 1 (Global Otsu)**: Global Otsu calculates $T = 186$, partitioning the image into bright background and dark foreground. Consequently, the person, clothing, hand, and black cup are merged into one large undifferentiated segment.
2. **Failure Mode 2 (Naive Dark Thresholding $I < 80$)**: While directly targeting intensities below 80 captures the black cup, it lacks spatial localization. As a result, it falsely captures dark hair at the top of the head and deep shadows in clothing folds.

7.3 Proposed Localized Composite Pipeline

The proposed pipeline resolves both failure modes by combining:

7.4 Python Implementation and Results

```

1 # 1. Failure Mode 1: Global Otsu Thresholding
2 failure_otstu = otsu_seg.copy()
3
4 # 2. Failure Mode 2: Naive Dark Intensity Thresholding ( $I < 80$ )
5 failure_dark_thresh = (gray < 80).astype(np.uint8) * 255
6
7 # 3. Proposed Localized Composite Pipeline:
8 seed_black_cup = (187, 230)
9 raw_cup_mask = region_growing(gray, seed_black_cup, threshold=25)
10
11 # Step 2: Morphological Refinement
12 kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
13 closed_cup_mask = cv2.morphologyEx(raw_cup_mask, cv2.MORPH_CLOSE, kernel)
14 final_cup_mask = cv2.morphologyEx(closed_cup_mask, cv2.MORPH_OPEN, kernel)
15
16 # Step 3: Object Retention - Retain only the black cup, replace rest with white
17 retained_black_cup = img_rgb.copy()
18 retained_black_cup[final_cup_mask == 0] = [255, 255, 255]
19
20 # Step 4: Object Disappearance - Inpaint the black cup to restore hand/scene

```

```

21 disappeared_bgr = cv2.inpaint(img_bgr, final_cup_mask, inpaintRadius=9, flags=cv2.
    INPAINT_TELEA)
22 disappeared_rgb = cv2.cvtColor(disappeared_bgr, cv2.COLOR_BGR2RGB)

```

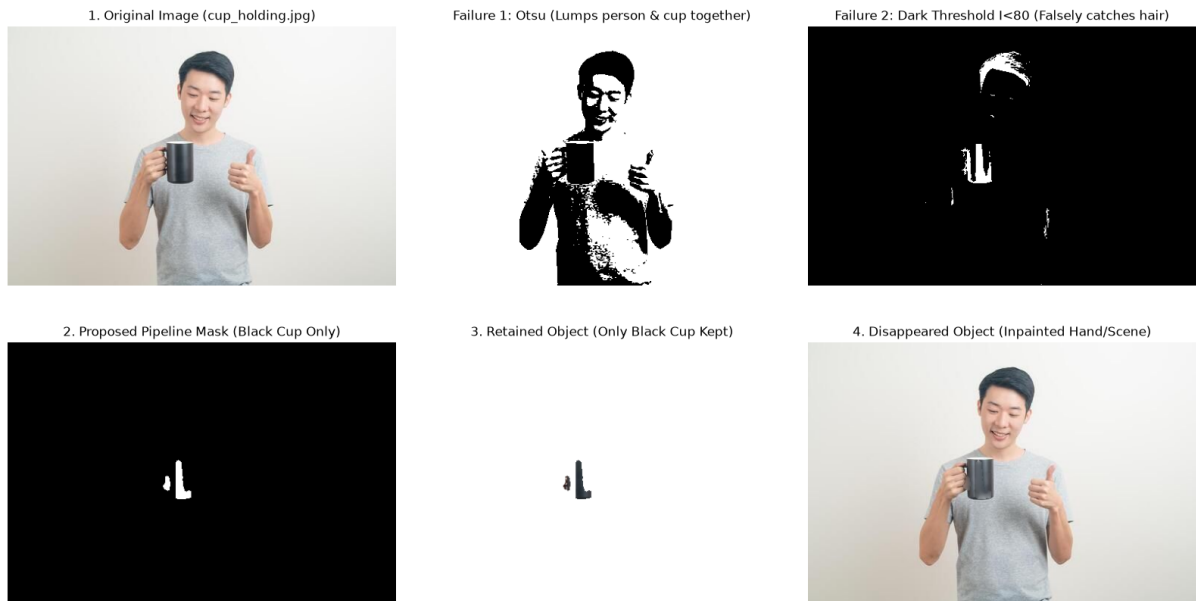


Figure 6: Part B pipeline results: (1) Original Image, (2) Failure 1: Otsu, (3) Failure 2: Naive Dark Threshold $I < 80$, (4) Proposed Pipeline Mask, (5) Retained Black Cup, and (6) Disappeared Cup via inpainting.

7.5 Analytical Discussion: Why the Composite Pipeline Succeeds

- **Spatial Constraint:** Region growing begins strictly inside the black cup ($y = 187, x = 230$), preventing false positives from distant dark features (e.g., hair and garments).
- **Exploiting Contrast Gradients:** The steep intensity drop between the black cup ($I \approx 48$) and the hand ($I \approx 180$) and gray wall ($I \approx 220$) guarantees clean edge arrest at $\Delta T = 25$.
- **Morphological Cleanup:** Elliptical closing eliminates inner specular holes, while opening severs delicate contact points with the fingers.
- **Seamless Inpainting:** The clean mask allows Telea inpainting to smoothly synthesize plausible skin and background textures, making the cup disappear naturally without blurring the hand posture.

8 Conclusion

This assignment comprehensively demonstrated spatial segmentation algorithms on `cup_holding.jpg`. While classical point and line masks isolate directional features and gradient operators (especially Sobel) provide continuous edge contours, they cannot produce closed semantic region partitions. Global thresholding (Otsu and Iterative) accurately separates foreground from background but fails to isolate sub-objects due to intensity overlap.

By designing a composite pipeline combining seeded region growing, calibrated similarity thresholds, and morphological refinement, we achieved precise segmentation of the black cup from the surrounding gray background and holding hand. Furthermore, applying fast-marching inpainting over the segmented mask successfully removed the object from the hand posture, establishing an effective workflow for object retention and disappearance in computer vision.